COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structures and Algorithm | **SECTION:** | CPE21S1 |
| **DATE PERFORMED:** | 4/8/2026 | **DATE SUBMITTED:** | 4/8/2026 |
| **NAME:** | Alvarez, Joshua Caleb P. | **INSTRUCTOR:** | Engr. Jimlord Quejado |

**ACTIVITY NO. _5.1__ Progress report hands on activity 5.1 Queues**

## 1. PROCEDURE OUTPUT

**ILOA CODE**

```cpp
//ALvarez, Joshua Caleb p.
#include<iostream>
#include<queue>

void display(std::queue<char>r);
void empty();

//
int main(){
    std::queue<char> q;

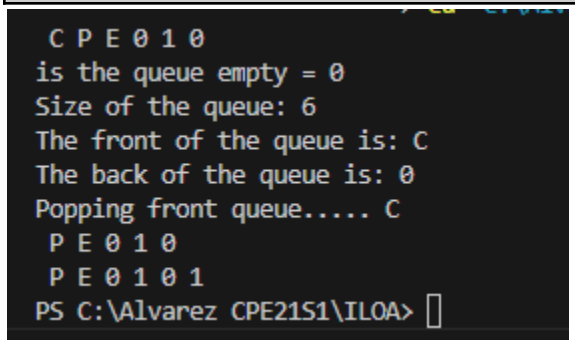
    q.push('C');
    q.push('P');
    q.push('E');
    q.push('0');
    q.push('1');
    q.push('0');

    //display
    display(q);

    //empty
    std::cout<<"is the queue empty = "<< q.empty()<<std::endl;
```

```
25      //size
26      std::cout<<"Size of the queue: "<< q.size() << std::endl;
27
28      //front
29      std::cout<<"The front of the queue is: "<< q.front()<<std::endl;
30
31      //back
32      std::cout<<"The back of the queue is: "<< q.back()<<std::endl;
33
34      //pop
35      std::cout << "Popping front queue..... " << q.front()<<std::endl;
36      q.pop();
37      //display
38      display(q);
39
40      //push another element
41      q.push('1');
42
43      //display
44      display(q);
45
46
47  }
48
49  void display(std::queue<char>r){
50      std::queue<char> c = r;
51
52       while(!c.empty()){
53           std::cout<<" "<<c.front();
54           c.pop();
55
56       }
57       std::cout<<"\n";
58
59   }
60
```

## ILOA OUTPUT




## Analysis and Conclusion

This code demonstrates a basic **queue in C++** using the `queue` library. The program creates a character queue and uses `push()` to add the letters of **"CPE 01"** into the queue. The `display()` function is used to show the elements, while `empty()` checks if the queue contains any elements. This activity helped me understand how the **FIFO (First-In, First-Out)** and how works and how basic queue functions are used in C++.

## ILOB CODE

main.cpp

```cpp
#include<iostream>
#include "queuell.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    std::cout<<"====================================="<<std::endl;
    std::cout<<"--------- QUEUE ELEMENTS ----------"<<std::endl;
    std::cout<<"====================================="<<std::endl;

    enqueue(&front, &back, 'J');
    std::cout<<front->data << " " << back->data<<std::endl;
    enqueue(&front, &back,'C' );
    std::cout<<front->data << " " << back->data<<std::endl;
    enqueue(&front, &back,'Z' );
    std::cout<<front->data << " " << back->data<<std::endl;
    enqueue(&front, &back,'I' );
    std::cout<<front->data << " " << back->data<<std::endl;
    enqueue(&front, &back,'E' );
    dequeue(&front, &back);
    std::cout<<front->data << " "<<back->data<<std::endl;
    std::cout<<std::endl;

    std::cout<<"\nDISPLAYING ALL OF THE ELEMENTS....\n\n";
    displayall(front);
    dequeue(&front, &back);
    displayall(back);

    return 0;
}
```

queuell.h

```
#ifndef QUEUELL_H
#define QUEUELL_H

template<typename T>
class qNode{
    public:
    T data;
    qNode *next;
};

//creating a new node
template<typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

template<typename T>
void enqueue(qNode<T>** frontPtr, qNode<T>** backPtr, T newData){
// create a new node
qNode<T>* newNode = new_node(newData);
//inserting to an empy queue
if((*frontPtr) == nullptr && (*backPtr) ==nullptr){
```

```
25  if((*frontPtr) == nullptr && (*backPtr) ==nullptr){
26  (*frontPtr) = newNode;
27  (*backPtr) = newNode;
28  }
29
30
31  //inserting an item into a non empty queue
32  (*backPtr) ->next = newNode;
33  (*backPtr) = newNode;
34
35
36  }
37  template<typename T>
38  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
39      //create a temporary to store the node to be deleted
40      qNode<T>* deleteNode = nullptr;
41      deleteNode = (*frontPtr);
42      //check if the queue is only 1 Node
43      if((*frontPtr)->next == nullptr && (*backPtr) -> next == nullptr){
44          (*frontPtr) = nullptr;
45          (*backPtr) = nullptr;
46          delete deleteNode;
47          return;
48      }
49      //deleting of the Node
50      (*frontPtr)=deleteNode->next;
```

```
53  }
54
55  //display all the elements in the list
56  template <typename T>
57  void displayall(qNode<T>* front)  {
58      if (front == nullptr){
59          std::cout<<"Queue is empty. \n";
60          return;
61      }
62      while (front != nullptr){
63          std::cout << front->data << " " << std:: endl;
64          front = front->next;
65      }
66      std::cout<<std::endl;
67  }
68
69  // return the front variable
70
71
72  #endif
```

## ILOB OUTPUT

```
PS C:\Alvarez CPE21S1\ILOB> cd "c:\Alvarez CPE21S1\ILOB\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
==================================
--------- QUEUE ELEMENTS ----------
==================================
J J
J C
J Z
J I
C E


DISPLAYING ALL OF THE ELEMENTS....
C
Z
I
E

E
```

```
[Running] cd "c:\Users\RYZEN 5\OneDrive\DSA PROJEC
PROJECTS\QUEUES\"elements
====================================
--------- QUEUE ELEMENTS ----------
====================================
J J
J C
J Z
J I
C E

DISPLAYING ALL OF THE ELEMENTS....

C Z I E
```

## ANALYSIS AND CONCLUSION

The output shows that the `qNode<T>` linked-list queue works properly using the FCFS (First-Come, First-Served) method. The first element added, `'J'`, was also the first one removed. The `front` and `back` pointers help keep track of the first and last elements in the queue. Using double pointers allows the `enqueue` and `dequeue` functions to update the actual pointers in `main()`, so the queue works correctly when adding and removing elements.

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**ILO C**: Solve problems using queue implementation
**Problem Title**: Shared Printer Simulation using Queues
**Problem Definition**: In this activity, we'll simulate a queue for a shared printer in an office. In any corporate office, Usually, the printer is shared across the whole floor in the printer room. All the computers in this room are connected to the same printer. But a printer can do only one printing job at any point in time, and it also takes some time to complete any job. In the meantime, some other user can send another print request. In such a case, a printer needs to store all the pending jobs somewhere so that it can take them up once its current task is done. Perform the following steps to solve the activity. Make sure that you include a screenshot of the source code for each. From the get-go: You must choose whether you are making a linked list or array implementation.
You may NOT use the STL.
1. Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages).
2. Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.
3. To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic strategy – first come, first served. Whoever submits the job first will be the first to get the job done.
4. Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one.
Defend your choice of internal representation: Why did you use arrays or linked lists?
Output Analysis:
- Provide the output after performing each task above.
- Include your analysis: focus on why you think the output is the way it is.

**ILOC CODE**

Job.h

```
ILOC > C Job.h > ...
1   #ifndef JOB_H
2   #define JOB_H
3
4   #include <string>
5
6   class Job {
7   public:
8       int jobID;
9       std::string userName;
10      int pages;
11
12      Job(int id, std::string name, int pageCount);
13  };
14
15  #endif
```

## ANAIYSIS FOR Job.h

The `Job.h` file contains the **Job** class, which is used to store the information of every printing job. It contains the job ID, the name of the user, and the number of pages to be printed. The constructor is used to assign values to these attributes when a new job is created. This file is important because it provides the basic structure and information that the printer queue will manage.

**CODE FOR printer.h**

ILOC > C printer.h > ...

```
1   #ifndef PRINTER_H
2   #define PRINTER_H
3
4   #include "Job.h"
5
6   class JobNode {
7   public:
8       Job job;
9       JobNode* next;
10
11      JobNode(Job newJob);
12  };
13
14  class Printer {
15  private:
16      JobNode* front;
17      JobNode* rear;
18
19  public:
20      Printer();
21
22      void addJob(int id, std::string name, int pages);
23      void processJob();
24      void displayJobs();
25      void processAllJobs();
26  };
27
28  #endif
```

## ANALYSIS FOR printer.h

The **`Printer.h`** file contains the structure of the printer queue. It includes the **`JobNode`** class, which connects each printing job to the next job using a pointer. It also contains the **`Printer`** class with the **`front`** and **`rear`** pointers. The **`front`** points to the first job in the queue, while the `rear` points to the last job. The function declarations for adding, processing, displaying, and processing all jobs are also placed in this file. This allows the printer to follow the FIFO principle of a queue.

**CODE FOR printer.cpp**

```cpp
//ALVAREZ,JOSHUA CALEB P.
#include <iostream>
#include <string>
#include "Printer.h"

Job::Job(int id, std::string name, int pageCount)
{
    jobID = id;
    userName = name;
    pages = pageCount;
}
JobNode::JobNode(Job newJob)
    : job(newJob), next(nullptr)
{
}

Printer::Printer()
{
    front = nullptr;
    rear = nullptr;
}

void Printer::addJob(int id, std::string name, int pages)
{
    Job newJob(id, name, pages);
```

```cpp
void Printer::addJob(int id, std::string name, int pages)
{
    Job newJob(id, name, pages);

    JobNode* newNode = new JobNode(newJob);

```

```cpp
    if (rear == nullptr)
    {
        front = newNode;
        rear = newNode;
    }
    else
    {
        rear->next = newNode;
        rear = newNode;
    }

    std::cout << "Job " << id << " added successfully." << std::endl;
    std::cout << "User: " << name << " | Pages: " << pages << std::endl;
    std::cout << std::endl;
}
```

```cpp
void Printer::displayJobs()
{
    if (front == nullptr)
    {
        std::cout << "No pending print jobs." << std::endl;
        return;
    }
    JobNode* current = front;
    std::cout << "----------------------------------------"<< std::endl;
    std::cout << "          PENDING PRINT JOBS"<< std::endl;
    std::cout << "----------------------------------------"<< std::endl;
    while (current != nullptr)
    {
        std::cout << "Job ID: "<< current->job.jobID<< std::endl;
        std::cout << "User: "<< current->job.userName<< std::endl;
        std::cout << "Pages: "<< current->job.pages<< std::endl;
        std::cout << "----------------------------------------"<< std::endl;

        current = current->next;
    }

```

```cpp
void Printer::displayJobs()
{
    if (front == nullptr)
    {
        std::cout << "No pending print jobs." << std::endl;
        return;
    }
    JobNode* current = front;
    std::cout << "----------------------------------------"<< std::endl;
    std::cout << "          PENDING PRINT JOBS"<< std::endl;
    std::cout << "----------------------------------------"<< std::endl;
    while (current != nullptr)
    {
        std::cout << "Job ID: "<< current->job.jobID<< std::endl;
        std::cout << "User: "<< current->job.userName<< std::endl;
        std::cout << "Pages: "<< current->job.pages<< std::endl;
        std::cout << "----------------------------------------"<< std::endl;

        current = current->next;
    }

    std::cout << std::endl;
}
```

```cpp
    std::cout << std::endl;
}

void Printer::processAllJobs()
{
    std::cout << "========================================"<< std::endl;
    std::cout << "       PROCESSING ALL PRINT JOBS"<< std::endl;
    std::cout << "========================================"<< std::endl;
    while (front != nullptr)
    {
        processJob();
    }
    std::cout << "All print jobs have been completed."<< std::endl;
}
```

## ANALYSIS FOR printer.cpp

The **`Printer.cpp`** file contains the actual implementation of the classes and functions declared in the header files. The `addJob()` function adds a new job to the rear of the queue, while **`processJob()`** removes the job from the front after it has been processed. **The `displayJobs()`** function traverses the linked list and displays all pending jobs. **The `processAllJobs()`** function continues processing until the queue becomes empty. The use of pointers, dynamic memory, `new`, and **`delete`** allows the queue to manage jobs dynamically.

**CODE FOR main.cpp**

```cpp
#include <iostream>
#include "Printer.h"
int main()
{
    Printer printer;

    std::cout << "========================================"<< std::endl;
    std::cout << "       SHARED PRINTER SIMULATION"<< std::endl;
    std::cout << "========================================"<< std::endl;
    std::cout << std::endl;
    // Adding Jobs
    std::cout << "ADDING PRINT JOBS..."<< std::endl;

    std::cout << std::endl;
    printer.addJob(101, "Joshua", 5);
    printer.addJob(102, "Carlos", 10);
    printer.addJob(103, "Maria", 3);
    printer.addJob(104, "Daniel", 8);
    // Display Current Queue
    std::cout << "CURRENT QUEUE:"<< std::endl;
    printer.displayJobs();
    // Process Two Jobs
    std::cout << "STARTING PRINTER..."<< std::endl;
    std::cout << std::endl;
    printer.processJob();
    printer.processJob();
    // Display Remaining Jobs
    std::cout << "REMAINING QUEUE:"<< std::endl;
    printer.displayJobs();

    // Process Remaining Jobs
    printer.processAllJobs();

    return 0;
}
```

**ANALYSIS FOR main.cpp**

The `main.cpp` file is responsible for running the shared printer simulation. It creates a `Printer` object and adds several jobs from different users. The program then displays the current queue, processes the first two jobs, displays the remaining jobs, and finally processes all remaining jobs. The output demonstrates the FIFO behavior of the queue because the jobs are processed in the same order that they were added.

**OUTPUT**

```
========================================
       SHARED PRINTER SIMULATION
========================================

ADDING PRINT JOBS...

Job 101 added successfully.
User: Joshua | Pages: 5

Job 102 added successfully.
User: Carlos | Pages: 10

Job 103 added successfully.
User: Maria | Pages: 3

Job 104 added successfully.
User: Daniel | Pages: 8
```

```
----------------------------------------
          PENDING PRINT JOBS
----------------------------------------
Job ID: 101
User: Joshua
Pages: 5
----------------------------------------
Job ID: 102
User: Carlos
Pages: 10
----------------------------------------
Job ID: 103
User: Maria
Pages: 3
----------------------------------------
Job ID: 104
User: Daniel
Pages: 8
----------------------------------------
```

```
STARTING PRINTER...

Processing Job 101...
User: Joshua
Pages: 5
Job 101 completed.

Processing Job 102...
User: Carlos
Pages: 10
Job 102 completed.

REMAINING QUEUE:
```

```
REMAINING QUEUE:
----------------------------------------
          PENDING PRINT JOBS
----------------------------------------
Job ID: 103
User: Maria
Pages: 3
----------------------------------------
Job ID: 104
User: Daniel
Pages: 8
----------------------------------------
```

```
----------------------------------------

========================================
       PROCESSING ALL PRINT JOBS
========================================
Processing Job 103...
User: Maria
Pages: 3
Job 103 completed.

Processing Job 104...
User: Daniel
Pages: 8
Job 104 completed.

All print jobs have been completed.
PS C:\Users\RYZEN 5\OneDrive\DSA PROJECTS\QUEUES\ILOC>
```

## Analysis and Conclusion

The output shows that the printer simulation works correctly using the FCFS method. The jobs were processed in the same order they were added to the queue. After the first two jobs were printed, the remaining jobs were still in the correct order. When all jobs were processed, the queue became empty. This shows that the queue is useful for a printer because it handles each job one at a time, starting with the job that was added first.

## CONCLUSION

The shared printer simulation successfully demonstrates how a **queue using a linked-list implementation** can be applied to a real-world situation. The program follows the **First-In, First-Out (FIFO)** principle, ensuring that the first user who submits a printing job is the first one to have the job processed. The linked-list implementation was chosen because it allows the number of jobs to increase or decrease dynamically without requiring a fixed queue size.

While creating the program, I encountered several challenges, especially with the **pointers, constructors, and queue operations**. Some errors occurred because the **`front`** and **`rear`** pointers were not properly initialized or updated when adding and removing jobs. I also had to check the **`next`** pointer of each node carefully to make sure that the linked list was connected correctly. To analyze the code, I checked each function separately, starting with the constructor, then the **`add`**, **`pop`**, and display functions. I also traced how the **`front`** and **`rear`** pointers changed after every operation. By testing different situations, such as adding multiple jobs and removing the last remaining job, I was able to identify and correct the errors.

Overall, the activity helped me understand not only queue operations such as **adding, processing, displaying, and removing elements**, but also how to debug and analyze a C++ program using **classes, pointers, linked lists, and dynamic memory allocation**.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

- This activity helped me understand how a queue works using a linked list. I encountered some errors with the pointers and the `front` and `rear`, so I checked the code step by step and fixed the problems. Despite from thatl, I learned how FIFO, linked lists, and pointers work together in C++.

Summary of lessons learned

**I learned how queues work using linked lists and the FIFO method. I also learned how to use pointers, classes, and basic queue operations in C++.**

COMPUTER ENGINEERING

QUEZON CITY

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **ChatGPT**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Code assistance, explanation of concepts, and grammar correction.**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **ChatGpt was used to help me understand and fix errors in the code and improve the wording of the written explanations.The code was reviewed, tested, and completed by me.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.